

Mass Assignment — Interview Prep Notes

1. Definition

- Occurs when app frameworks auto-bind client-supplied request params directly to internal objects/models/DB fields without allowlist filtering
- Attacker adds/modifies fields not exposed in UI/API contract but present in backend model (e.g., `role`, `isAdmin`, `balance`, `verified`)
- Root cause: **trust boundary violation** — binding layer treats all input keys as legitimate
- Related concept: **Over-posting** (same vuln, different name in .NET ecosystem)

2. OWASP / CWE

- **OWASP API Security Top 10 2023:** API3:2023 – Broken Object Property Level Authorization (merged old API3 Excessive Data Exposure + API6 Mass Assignment)
- **OWASP Web Top 10 2021:** A08:2021 – Software and Data Integrity Failures (secondary mapping; some map to A01 Broken Access Control depending on impact)
- **CWE-915:** Improperly Controlled Modification of Dynamically-Determined Object Attributes
- **CWE-620:** Unverified Password Change (if mass assignment affects auth fields)

3. Common Variants

- **Privilege escalation via role/permission field** (`role=admin`, `isAdmin=true`)
- **Price/quantity tampering** (e-commerce — `price`, `discount`)
- **Account state tampering** (`isVerified`, `accountBalance`, `credit_limit`)
- **Nested object mass assignment** (JSON nested objects bound recursively)
- **Array-based mass assignment** (bulk update endpoints binding array of objects)
- **ORM-level mass assignment** (ActiveRecord `update_attributes`, Mongoose `.save()`, EF `.Bind()`)

4. Attack Surface / Entry Points

- Registration/signup endpoints
- Profile update / PATCH-PUT endpoints
- Checkout/order creation APIs
- Admin-adjacent APIs reused by regular users (shared DTO/model)
- GraphQL mutations (input types often map 1:1 to DB schema)
- Bulk import/CSV upload → object binding

- Webhooks/internal APIs with relaxed validation

5. Testing Workflow

Step	Goal	Manual	Burp	ZAP	Nuclei	Payload/Rationale
Recon — map object schema	Identify all model fields (not just UI-exposed)	Diff JS bundle, API docs, GraphQL introspection, error messages leaking field names	Passive crawl + JS file analysis	Spider + AJAX spider	No (recon only)	Look for <code>id</code> , <code>role</code> , <code>isAdmin</code> , <code>price</code> , <code>status</code> , <code>verified</code>
Baseline request capture	Establish normal request/response	Send legit request via UI	Proxy → HTTP history	Proxy → History tab	N/A	Normal signup/update payload
Field injection	Verify if extra param is bound	Manually add field to JSON body	Repeater — add key-value pairs	Manual Request Editor	Custom template with fuzzed keys	<code>"role": "admin"</code> , <code>"isAdmin": true</code> , <code>"price": 0.01</code>
Response diffing	Confirm binding accepted (not just echoed)	Compare object state via GET after PATCH	Repeater + Comparer	Compare tab	N/A	Check DB/GET response reflects change, not just 200 OK
Blind confirmation	Confirm privilege change took effect	Re-auth as victim, check access level	Session handling rules to test w/ 2nd account	Auth context switch	N/A	Access admin-only endpoint post-injection
Automated fuzzing	Brute force hidden field names	Wordlist of common sensitive fields	Intruder — cluster bomb on JSON keys	Fuzzer add-on	Custom YAML w/ field wordlist	<code>role</code> , <code>isAdmin</code> , <code>admin</code> , <code>is_staff</code> , <code>permissions</code> , <code>credit</code> , <code>balance</code> , <code>verified</code> , <code>status</code> , <code>price</code> , <code>discount</code> , <code>user_id</code>
Nested/array test	Test nested object binding	Manually nest object in JSON	Repeater	Manual editor	N/A	<code>{"user": {"role": "admin"}}</code>

- **Nuclei:** no strong built-in templates (logic flaw, needs custom fuzzing template w/ matcher on response field reflection); mainly useful for wordlist-driven param discovery, not core testing

6. Evidence to Collect

- Original request/response (baseline, unauthorized field absent)
- Modified request with injected field + response showing 200/updated resource
- Follow-up GET/API call proving persisted state change in DB
- Screenshot/API response showing privilege change (e.g., admin panel access)
- HTTP history timeline (request → effect) for timeline in report

7. Good PoC

- Step 1: Register normal user, capture request
- Step 2: Add `"role": "admin"` (or relevant sensitive field) to request body
- Step 3: Show response/GET confirms field persisted
- Step 4: Demonstrate **business impact** — log in and access admin-restricted endpoint/resource
- Full loop: injection → persistence → privilege/impact demonstrated, not just "field accepted"

8. Attack Chains

- Mass Assignment → Privilege Escalation → Full Admin Takeover
- Mass Assignment → IDOR (set `user_id` / `owner_id` field to victim's ID)
- Mass Assignment → Price Manipulation → Financial Fraud
- Mass Assignment → Account Verification Bypass → Auth Bypass
- Mass Assignment (self-registration) → Stored XSS (if injecting fields render unsanitized elsewhere)

9. False Positives

- Field accepted in request but silently ignored by backend (no persistence) — must verify via GET/DB state
- Field bound internally but overwritten later in business logic (post-validation reset)
- Field is legitimately user-controllable (not sensitive) — confirm sensitivity before flagging
- Response reflects submitted JSON as-is (echo) without actual persistence

10. Prerequisites for Exploitability

- Framework auto-binding enabled (ActiveRecord, Mongoose `.save(req.body)`, EF, Django ModelForm w/o `fields=`)
- No allowlist/DTO pattern enforced at API layer
- Sensitive field present in same model/schema as user-editable fields

- Endpoint accepts extra/unexpected JSON keys without strict schema validation (no `additionalProperties: false`)

11. Attacker Impact

- Privilege escalation (user → admin)
- Unauthorized data modification (price, balance, discount, quota)
- Bypass of business/workflow controls (approval states, verification flags)
- Horizontal access via ownership field overwrite (IDOR overlap)

12. CIA Impact Table

Variant	C	I	A	Example CVSS Vector (v3.1)
Role/Privilege escalation	High	High	Low	<code>AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:L</code> (9.9)
Price/financial tampering	Low	High	None	<code>AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N</code> (6.5)
Ownership/IDOR overwrite	High	High	None	<code>AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:N</code> (9.6)
Account state (verified/status)	Low	Medium	None	<code>AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N</code> (5.4)

13. Typical CVSS

- Range: **5.4 (Medium)** to **9.9 (Critical)** depending on scope (S:C if crosses privilege boundary) and field sensitivity
- Privilege escalation to admin scenario → Critical, near-max score
- Minor state tampering (non-financial, non-auth) → Medium

14. Business Impact (Financial App Context)

- Direct fraud: balance/credit_limit manipulation → monetary loss
- Regulatory: PCI-DSS (unauthorized modification of payment-related fields) violation
- SOX/financial controls: integrity of transaction records compromised — audit failure
- Reputational: privilege escalation in banking app → regulatory reporting mandate, customer trust loss
- GDPR: unauthorized modification of PII fields (e.g., changing linked account/beneficiary data) → integrity breach notification obligation

15. Detection Signals/IOCs

- Unexpected fields in request logs (fields not in documented API schema)

- WAF/API gateway logs showing rejected/anomalous JSON keys
- Sudden privilege changes without corresponding admin action in audit log
- DB records with fields modified outside expected business workflow (e.g., `verified=true` without verification event)
- Mismatch between UI form fields and actual request body captured in proxy logs

16. Root Cause (Code Level)

Vulnerable (Node/Mongoose):

```
const user = new User(req.body); // binds ALL fields incl. role
await user.save();
```

Vulnerable (Rails, old-style):

```
User.new(params[:user]) # no strong params
```

Vulnerable (C#/EF):

```
var user = new User();
TryUpdateModel(user, "user", ["*"]); // wildcard binds all
```

- Common pattern: entire `req.body` / `params` passed directly to model constructor/update method
- Backend model = superset of fields (includes sensitive/internal); no separation between **DTO (client-facing)** and **domain model**

17. Remediation / Secure Design

- **Allowlist (whitelist) binding** — explicitly define bindable fields per endpoint
- **DTO pattern** — separate request/response objects from DB models; map explicitly
- **Framework-native protections:**
 - Rails: `params.require(:user).permit(:name, :email)` (Strong Parameters)
 - Django: explicit `fields = [...]` in ModelForm/Serializer (DRF)
 - .NET: `[Bind(Include="Name,Email")]` or explicit DTO + AutoMapper w/ ignore rules
 - Mongoose: explicit field selection before `.save()`, avoid direct `new Model(req.body)`
 - Spring: `@JsonIgnore` on sensitive fields + separate request DTO
- Server-side authorization check on sensitive field changes (e.g., role change requires admin-level auth, not just authenticated)
- Schema validation with `additionalProperties: false` (JSON Schema/OpenAPI enforcement)

18. Compensating Controls

- API Gateway/WAF rule blocking known sensitive field names in request body
- Strict JSON schema validation middleware (reject unknown keys) as a stop-gap
- Logging/alerting on presence of sensitive field names in incoming requests
- Post-write validation job comparing actual vs expected field deltas (detective control)
- Rate limiting + anomaly detection on profile/account update endpoints

19. Recurrence Implications (SDLC Signal)

- Indicates **no shared DTO/serialization standard** across services — systemic, not one-off
- Suggests missing **secure API design review** gate pre-merge
- Points to **inconsistent framework usage** (teams bypassing built-in protections like Strong Params/DRF serializers)
- Signals absence of **schema-first API design** (OpenAPI contract not enforced at runtime)

20. Upstream Improvements

- **SAST rule (Semgrep taint-mode)**: flag direct `new Model(req.body)` / `.save(req.body)` / `params[:model]` without permit/serializer
- **Linters/framework enforcement**: mandate DRF serializers, Rails Strong Params, explicit DTOs — fail CI if bypassed
- **API contract testing**: OpenAPI schema validation in CI (reject undocumented fields)
- **Design checklist item**: "Does this endpoint use explicit field allowlisting?" in PR template / threat model
- **Shared wrapper library** (per Akshat's IDOR pattern) — central serialization layer enforcing allowlist across all services

21. Sample Triage Response

"Confirmed Mass Assignment on `PATCH /api/v1/users/{id}`. Backend binds full request body to User model without field-level allowlisting. Injected `role:admin` field persisted successfully and granted admin panel access using low-privilege account. Severity: Critical (CVSS 9.9) — privilege escalation, scope changed. PoC and request/response evidence attached. Recommend immediate patch via Strong Parameters/DTO allowlist on affected endpoint(s); requesting audit of all PATCH/POST endpoints using similar binding pattern."

22. Sample Developer Remediation Note

"Root cause: `User.new(params[:user])` binds all incoming fields directly to model, including `role`. Fix: replace with Strong Parameters — `params.require(:user).permit(:name, :email)`. Do not add `:role` to permitted list; role changes must go through separate admin-only endpoint with explicit authorization check. Apply same pattern audit across all `.new(req.body)` / `.save(req.body)` usages repo-wide — recommend Semgrep rule to catch regressions in CI."

23. Real-World Examples

- **GitHub (2012):** Public mass assignment vuln — attacker added SSH key to Rails Engine Yard org via public key upload form due to unprotected `params[:user]` binding (pre Strong Parameters default) — became the landmark case that pushed Rails to make Strong Parameters default in Rails 4
- **Venmo API:** repeated bug bounty reports of mass assignment allowing manipulation of transaction visibility/privacy fields via public API
- **Common bug bounty pattern:** HackerOne reports on SaaS platforms — `"role": "owner"` or `"plan": "enterprise"` injectable via org invite/signup endpoints, granting elevated tier/privilege for free